
C: Como Programar

6ª Edição · Paul Deitel & Harvey Deitel

Capítulo 2

Introdução à Programação em C

Exercícios de Autorrevisão

Exercícios 2.1 e 2.2 – Completamente resolvidos

Abordagem incremental: Capítulos 1 e 2

Fundamentos Acumulados até o Capítulo 2

Neste capítulo, você escreveu seus primeiros programas completos em C. Construindo sobre a base do Capítulo 1 (hardware, software, linguagens e ambiente de desenvolvimento), o Capítulo 2 introduz: a estrutura de um programa C (**main**, chaves, ponto e vírgula), as funções **printf** e **scanf**, variáveis e tipos (**int**), operadores aritméticos, precedência e a instrução **if**. Os exercícios de autorrevisão consolidam esses fundamentos essenciais.

Boa prática de programação

Os conceitos do Capítulo 2 são a espinha dorsal de toda programação em C. Domine **printf**, **scanf**, variáveis, aritmética e **if** agora, pois eles serão usados em **todo** programa que você escreverá durante este curso.

Contents

1	Exercício 2.1 – Preenchimento de Lacunas	2
2	Exercício 2.2 – Verdadeiro ou Falso	8

1. Exercício 2.1 – Preenchimento de Lacunas

Enunciado

Preencha os espaços nas seguintes frases:

- a) Todo programa em C inicia sua execução pela função _____.
- b) O(a) _____ inicia o corpo de cada função, e o(a) _____ encerra o corpo de cada função.
- c) Toda instrução termina com um(a) _____.
- d) A função _____ da biblioteca-padrão exibe informações na tela.
- e) A sequência de escape `\n` representa o caractere de _____, que faz com que o cursor se posicione no início da próxima linha na tela.
- f) A função _____ da biblioteca-padrão é usada para obter dados do teclado.
- g) O especificador de conversão _____ é usado em uma string de controle de formato de `scanf` para indicar que um inteiro será digitado, e em uma string de controle de formato de `printf` para indicar que um inteiro será exibido.
- h) Sempre que um novo valor é colocado em uma posição da memória, ele anula o valor que ocupava essa mesma posição anteriormente. Esse processo é chamado de _____.
- i) Quando um valor é lido de uma posição da memória, o valor nessa posição é preservado; esse processo é chamado de _____.
- j) A instrução _____ é usada na tomada de decisões.

Item (a) – A função que inicia a execução

Resposta detalhada**Resposta:** `main`**Explicação:**

Todo programa em C começa e organiza sua execução a partir da função `main`. Essa é uma regra absoluta da linguagem: independentemente de quantas funções um programa tenha, a execução sempre se inicia em `main`. A estrutura mínima de um programa C é:

```
1  /* Estrutura minima de um programa C */
2  #include <stdio.h>
3
4  int main( void )
5  {
6      /* instrucoes do programa */
7      return 0;    /* indica conclusao com sucesso */
8  } /* fim da funcao main */
```

- `int` → indica que `main` retorna um valor inteiro ao sistema operacional
- `void` → indica que `main` não recebe argumentos nessa forma simples
- `return 0;` → valor 0 sinaliza ao SO que o programa terminou com sucesso

Item (b) – Os delimitadores do corpo de uma função**Resposta detalhada****Resposta:** **chave à esquerda** (`{`) inicia / **chave à direita** (`}`) encerra**Explicação:**

Em C, o corpo de toda função é delimitado por um par de chaves. A **chave à esquerda** `{` marca o início do bloco de código da função, e a **chave à direita** `}` marca seu fim. Todo o código entre essas chaves compõe o *corpo* da função.

```
1  int main( void )
2  {          /* <- chave a esquerda: inicio do corpo */
3      printf( "Bem-vindo a C!\n" );
4      return 0;
5  }          /* <- chave a direita: fim do corpo */
```

O par de chaves com o código entre eles é chamado de **bloco**. Blocos serão muito importantes ao estudarmos estruturas de controle nos Capítulos 3 e 4.

Item (c) – O terminador de instrução**Resposta detalhada****Resposta:** **ponto e vírgula** (`;`)**Explicação:**

Em C, toda instrução executável deve ser encerrada com um **ponto e vírgula**, que é chamado de *terminador de instrução*. Exemplos:

```
1 printf( "Ola, mundo!\n" );    /* instrucao de saida --
    termina com ; */
2 int x;                        /* declaracao -- termina
    com ; */
3 x = 5 + 3;                    /* instrucao de
    atribuicao -- termina com ; */
```

Erro comum de programação

Esquecer o ponto e vírgula ao final de uma instrução é um dos erros de sintaxe mais comuns em C. O compilador emitirá uma mensagem de erro, geralmente na linha *seguinte* à instrução onde o ; está faltando, o que pode ser confuso.

Item (d) – A função que exibe informações na tela

Resposta detalhada

Resposta: `printf`

Explicação:

A função `printf` (*print formatted*) da biblioteca-padrão `<stdio.h>` é responsável por exibir informações formatadas na tela (no fluxo `stdout`). Sua sintaxe geral é:

```
1 printf( "string de controle de formato", arg1, arg2,
    ... );
```

Exemplos práticos:

```
1 printf( "Bem-vindo a C!\n" );    /* texto
    simples */
2 printf( "O valor e: %d\n", x );    /* exibe
    inteiro x */
3 printf( "Soma = %d\n", a + b );    /* calcula e
    exibe */
4 printf( "%d + %d = %d\n", a, b, a+b ); /* multiplos
    valores */
```

Item (e) – A sequência de escape \n

Resposta detalhada

Resposta: `nova linha` (*newline*)

Explicação:

A sequência de escape `\n` representa o caractere de **nova linha**. Quando `printf` encontra `\n` em uma string, move o cursor para o *início da próxima linha* na tela.

O caractere de barra invertida `\` é chamado de **caractere de escape**. Combinado com o próximo caractere, forma uma **sequência de escape** com significado especial. Outras sequências úteis:

Sequência	Significado
<code>\n</code>	nova linha
<code>\t</code>	tabulação horizontal
<code>\\</code>	barra invertida literal
<code>\"</code>	aspas duplas literais

Item (f) – A função que lê dados do teclado

Resposta detalhada

Resposta: `scanf`

Explicação:

A função `scanf` (*scan formatted*) lê dados formatados da entrada-padrão (`stdin`), que normalmente é o teclado. Sua sintaxe geral é:

```
1 scanf( "string de controle de formato", &variavel1, &
    variavel2, ... );
```

O símbolo `&` antes de cada variável é o **operador de endereço**, que informa à `scanf` onde na memória armazenar o valor lido. Exemplo:

```
1 int idade;
2 printf( "Digite sua idade: " );
3 scanf( "%d", &idade );    /* le um inteiro e armazena
    em idade */
```

Erro comum de programação

Esquecer o `&` antes de uma variável em `scanf` é um dos erros mais frequentes de programadores iniciantes. Sem o `&`, o programa pode causar uma “falha de segmentação” (acesso inválido à memória).

Item (g) – O especificador de conversão para inteiros

Resposta detalhada

Resposta: `%d`

Explicação:

O especificador de conversão `%d` é usado para indicar um valor inteiro decimal (a letra `d` vem de *decimal integer*). Ele funciona tanto em `scanf` (para ler um inteiro) quanto em `printf` (para exibir um inteiro):

```
1 int num;
2 scanf( "%d", &num );           /* le um inteiro do
   teclado */
3 printf( "Voce digitou: %d\n", num ); /* exibe o
   inteiro na tela */
```

O % em uma string de controle de formato sinaliza ao `scanf`/`printf` que um **especificador de conversão** está começando. Tudo que vem após % até o caractere de formato especifica o tipo e o formato do dado.

Item (h) – Colocar um novo valor em uma posição de memória

Resposta detalhada

Resposta: destrutivo

Explicação:

Da Seção 2.4: quando um novo valor é colocado em uma posição de memória, ele **substitui (destrói)** o valor que existia anteriormente naquela posição. Por isso, essa operação é chamada de **destrutiva**.

Exemplo passo a passo:

```
1 int x;
2 x = 10;    /* memoria: x = 10 */
3 x = 25;    /* memoria: x = 25 -- o valor 10 foi
   destruido! */
4 x = x + 1; /* memoria: x = 26 -- o valor 25 foi
   destruido! */
```

Na instrução `x = x + 1`, o computador primeiro *lê* o valor de `x` (operação não destrutiva), calcula `x + 1`, e então *armazena* o resultado em `x` (operação destrutiva – o valor anterior é perdido).

Item (i) – Ler um valor sem alterar a memória

Resposta detalhada

Resposta: não destrutivo

Explicação:

Quando um valor é *lido* de uma posição de memória para ser usado em um cálculo ou exibição, o valor nessa posição é **preservado**. Por isso, a operação de leitura é chamada de **não destrutiva**.

Exemplo:

```
1 int a = 5, b = 3;
2 int soma = a + b;    /* le a (=5) e b (=3) -- nao
   destrutivo */
3                      /* a ainda e 5, b ainda e 3 depois
                       do calculo */
```

```
4 printf( "%d %d %d\n", a, b, soma ); /* imprime: 5 3 8 */
```

A distinção entre operações destrutivas e não destrutivas é fundamental para entender o comportamento de variáveis e a instrução de atribuição em C.

Item (j) – A instrução de tomada de decisão

Resposta detalhada

Resposta: `if`

Explicação:

A instrução `if` (introduzida na Seção 2.6) permite que um programa tome decisões com base na verdade ou falsidade de uma **condição**. Sua forma básica é:

```
1 if ( condicao ) {
2     /* executado apenas se condicao for verdadeira (
3         nao zero) */
4     printf( "A condicao e verdadeira!\n" );
5 }
```

Condições são formadas com os **operadores relacionais** (<, >, <=, >=) e os **operadores de igualdade** (==, !=).

Erro comum de programação

O erro mais clássico com `if` é confundir o operador de igualdade `==` com o operador de atribuição `=`. O código `if (x = 5)` *não* verifica se `x` é igual a 5 – ele *atribui* 5 a `x`, e a condição sempre será verdadeira! Leia `==` como “dois iguais” e `=` como “recebe”.

2. Exercício 2.2 – Verdadeiro ou Falso

Enunciado

Indique se cada uma das sentenças a seguir é **verdadeira** ou **falsa**. Explique sua resposta no caso de alternativas *falsas*.

Item (a) – `printf` sempre começa no início de uma nova linha

FALSO

Explicação: A função `printf` começa a imprimir *onde o cursor estiver posicionado*, e isso pode ser em qualquer posição em uma linha. Por exemplo, se você chamar `printf("Ola ")` e depois `printf("Mundo!")`, a saída será `Ola Mundo!` tudo na mesma linha. Somente o caractere `\n` move o cursor para o início da próxima linha.

Item (b) – Comentários fazem o computador imprimir o texto na tela

FALSO

Explicação: Comentários em C (`/* ... */` ou `//`) são completamente *ignorados* pelo compilador – eles não geram nenhum código de máquina e não causam nenhuma ação durante a execução do programa. Sua única finalidade é **documentar o código** e melhorar sua legibilidade para os humanos.

```
1  /* Este texto NUNCA aparecera na tela -- e apenas um
   comentario */
2  printf( "Este texto SIM aparece na tela.\n" );
```

Item (c) – `\n` move o cursor para o início da próxima linha

VERDADEIRO

Explicação: A sequência de escape `\n` representa o caractere de *nova linha*. Quando o `printf` a encontra, posiciona o cursor no início da próxima linha da tela. É exatamente isso que a torna tão essencial em todo programa C.

Item (d) – Todas as variáveis precisam ser declaradas antes de serem usadas

VERDADEIRO

Explicação: Em C padrão (ANSI C), toda variável deve ser declarada com seu nome e tipo *antes* de ser usada. Além disso, as declarações devem aparecer **após a chave à esquerda de uma função e antes de quaisquer instruções executáveis**. Tentar usar uma variável não declarada causa um erro de compilação.

Item (e) – Todas as variáveis precisam receber um tipo ao serem declaradas

VERDADEIRO

Explicação: Em C, toda declaração de variável especifica obrigatoriamente um *tipo de dado*. Neste capítulo, usamos o tipo `int` para inteiros. Outros tipos que estudaremos mais adiante incluem `float`, `double` e `char`. O tipo informa ao compilador quanto espaço de memória reservar e como interpretar os bits armazenados naquela posição.

Item (f) – C considera `numero` e `NuMeRo` idênticas

FALSO

Explicação: C é uma linguagem **sensível a maiúsculas e minúsculas** (*case-sensitive*). Os identificadores `numero` e `NuMeRo` são tratados como variáveis *completamente diferentes*. Em C, cada letra conta: `Main` não é o mesmo que `main`, por exemplo – e isso causaria um erro de compilação.

Boa prática de programação

Por convenção Deitel, nomes de variáveis simples devem começar com letra minúscula. Identificadores iniciados com maiúscula terão significado especial em C++ (classes), que estudaremos nos Capítulos 15–24.

Item (g) – Declarações podem aparecer em qualquer parte do corpo de uma função

FALSO

Explicação: Em C padrão (ANSI C), as declarações de variáveis devem aparecer **após a chave à esquerda que abre o corpo da função e antes de quaisquer instruções executáveis**. Inserir uma declaração após uma instrução executável (como um `printf`) causa um *erro de sintaxe* na maioria dos compiladores C padrão.

```
1  int main( void )
2  {
3      int a;                /* OK: declaracao antes das
                             instrucoes */
4      printf( "ok\n" );    /* instrucao executavel */
5      int b;                /* ERRO em C padrao:
                             declaracao apos instrucao */
6      return 0;
7  }
```

Nota: C99 e C++ permitem declarações em qualquer ponto, mas neste livro seguimos o padrão C clássico.

Item (h) – Argumentos de printf precisam ser precedidos por &

FALSO

Explicação: O operador de endereço & é obrigatório nos argumentos de `scanf` (para que ela saiba *onde* armazenar o valor lido), mas **não** deve ser usado nos argumentos de `printf` (que apenas *lê* o valor da variável para exibi-lo).

```

1  int x = 42;
2  printf( "%d\n", x );      /* CORRETO: sem & em printf */
3  printf( "%d\n", &x );    /* ERRO: & em printf e
   incorreto */
4  scanf( "%d", &x );       /* CORRETO: com & em scanf
   */
5  scanf( "%d", x );        /* ERRO: sem & em scanf e
   perigoso */

```

As exceções a essa regra (relacionadas a arrays e ponteiros) serão discutidas nos Capítulos 6 e 7.

Item (i) – O operador módulo só pode ser usado com operandos inteiros

VERDADEIRO

Explicação: O operador módulo % produz o resto da divisão inteira e só é válido com operandos do tipo `int` (e outros tipos inteiros). Tentar usá-lo com números de ponto flutuante (`float`, `double`) causa um erro de compilação. Exemplo:

```

1  int a = 17, b = 5;
2  printf( "Resto: %d\n", a % b ); /* Resultado: 2 --
   CORRETO */
3  /* double x = 17.0 % 5.0; -- ERRO: % nao funciona com
   double */

```

Item (j) – Os operadores *, /, %, + e - têm o mesmo nível de precedência

FALSO

Explicação: Em C, os operadores aritméticos possuem **dois** níveis de precedência distintos (Seção 2.5):

Nível	Operadores	Avaliação
Maior	* / %	da esquerda para a direita
Menor	+ -	da esquerda para a direita

Portanto, $2 + 3 * 4$ é avaliado como $2 + 12 = 14$ (não como $5 * 4 = 20$), porque `*` tem maior precedência que `+`.

Item (k) – Nomes com mais de 31 caracteres são idênticos em todos os sistemas C

FALSO

Explicação: O padrão C garante que **apenas os primeiros 31 caracteres** de um identificador precisam ser reconhecidos pelos compiladores. Portanto, dois identificadores que diferem apenas após o 31º caractere podem ser tratados como *idênticos* por alguns compiladores, enquanto outros com suporte maior podem distingui-los. Os nomes do enunciado `esteéumnúmerosuperhiperlongo1234567` e `esteéumnúmerosuperhiperlongo1234568` diferem apenas no 44º caractere – isso pode causar problemas de portabilidade.

Boa prática de programação

Use identificadores com no máximo 31 caracteres para garantir portabilidade entre todos os compiladores C padrão.

Item (l) – Um programa que exibe três linhas precisa de três `printf`

FALSO

Explicação: Uma única instrução `printf` pode exibir múltiplas linhas usando várias seqüências de escape `\n`:

```
1  /* Uma instrucao printf exibindo tres linhas */
2  printf( "Linha 1\nLinha 2\nLinha 3\n" );
```

```
Linha 1
Linha 2
Linha 3
```

Cada `\n` na string de controle de formato gera um salto de linha.

Gabarito Rápido – Capítulo 2: Autorrevisão

Respostas Resumidas

Exercício 2.1:

- a) `main`
- b) chave à esquerda (`{`); chave à direita (`}`)
- c) ponto e vírgula (`;`)
- d) `printf`
- e) nova linha (*`newline`*)
- f) `scanf`
- g) `%d`
- h) destrutivo
- i) não destrutivo
- j) `if`

Exercício 2.2 – Verdadeiro/Falso:

- a) **Falso** – `printf` imprime onde o cursor estiver
- b) **Falso** – comentários são ignorados pelo compilador
- c) **Verdadeiro**
- d) **Verdadeiro**
- e) **Verdadeiro**
- f) **Falso** – C é sensível a maiúsculas e minúsculas
- g) **Falso** – declarações devem vir antes das instruções executáveis
- h) **Falso** – `&` é usado em `scanf`, não em `printf`
- i) **Verdadeiro**
- j) **Falso** – `*`, `/`, `%` têm precedência maior que `+`, `-`
- k) **Falso** – pode haver diferença após os 31 primeiros caracteres
- l) **Falso** – um único `printf` pode exibir várias linhas